CS1200: Intro. to Algorithms and their Limitations	Prof. Anurag Anshu

Lecture 20: NP-completeness, P vs NP

Harvard SEAS - Spring 2026 *2026-15-04*

1 Announcements

- Anurag's OH today 12:45-1:45 pm
- Next SRE Mon April 20.

Recommended Reading:

- Hesterberg–Vadhan 18
- MacCormick, 12.0 - 12.3, 13

2 $\mathsf{NP}_{\mathsf{search}}$-completeness

Although it is widely conjectured, we unfortunately do not know how to prove that $\mathsf{NP}_{\mathsf{search}} \not\subseteq \mathsf{P}_{\mathsf{search}}$. As we will see in the next lecture, this is an equivalent formulation of the famous P vs. NP problem, considered one of the most important open problems in computer science and mathematics. However, even without resolving the P vs. NP conjecture, we can give strong evidence that problems are not solvable in polynomial time by showing that they are $\mathsf{NP}_{\mathsf{search}}$-*complete*:

Definition 2.1 (NP-completeness, search version)**.** A problem Γ is $\mathsf{NP}_{\mathsf{search}}$-*complete* if:

1. Γ is in $\mathsf{NP}_{\mathsf{search}}$
2. Γ is $\mathsf{NP}_{\mathsf{search}}$-*hard*: For every computational problem $\Pi \in \mathsf{NP}_{\mathsf{search}}$, $\Pi \leq_p \Gamma$.

We can think of the NP-complete problems as the "hardest" problems in NP. Indeed:

Proposition 2.2. *Suppose* Γ *is* $\mathsf{NP}_{\mathsf{search}}$-*complete. Then* $\Gamma \in \mathsf{P}_{\mathsf{search}}$ *iff* $\mathsf{NP}_{\mathsf{search}} \subseteq \mathsf{P}_{\mathsf{search}}$.

Proof.

$\square$

In other words, if any $\mathsf{NP}_{\mathsf{search}}$-complete problem is in $\mathsf{P}_{\mathsf{search}}$, then all problems in $\mathsf{NP}_{\mathsf{search}}$ are in $\mathsf{P}_{\mathsf{search}}$.

3 $\mathsf{NP}_{\mathsf{search}}$-complete problems

Remarkably, there not only exist $\mathsf{NP}_{\mathsf{search}}$-complete problems, but some of them are quite natural. The first one we consider is SAT:

Theorem 3.1 (Cook–Levin Theorem)**.** SAT *is* $\mathsf{NP}_{\mathsf{search}}$*-complete.*

This can be interpreted as strong evidence that SAT is not solvable in polynomial time. If it were, then *every* problem in $\mathsf{NP}_{\mathsf{search}}$ would be solvable in polynomial time. We will take the Cook–Levin Theorem on faith for now, and may sketch a proof later.

Once we have one $\mathsf{NP}_{\mathsf{search}}$-complete problem, we can get others via reductions from it.

Theorem 3.2. 3SAT *is* $\mathsf{NP}_{\mathsf{search}}$*-complete.*

Proof. The proof follows in two steps.

1. 3SAT is in $\mathsf{NP}_{\mathsf{search}}$:

2. 3SAT is $\mathsf{NP}_{\mathsf{search}}$-hard: Since every problem in $\mathsf{NP}_{\mathsf{search}}$ reduces to SAT (by Theorem 3.1), all we need to show is SAT $\leq_p$ 3SAT (since reductions compose).

The reduction algorithm from SAT to 3SAT has the following components. First, we give an algorithm R which takes a SAT instance φ to a 3SAT instance φ'.

$$\text{SAT instance } \varphi \xrightarrow{\text{polytime R}} \text{3SAT instance } \varphi'$$

Then we feed the instance φ' to our 3SAToracle and obtain a satisfying assignment α' to φ' or $\perp$ if none exists. If we get $\perp$ from the oracle, we return $\perp$, else we transform α' into a satisfying assignment to φ using another algorithm S.

$$\text{SAT assignment } \alpha \xleftarrow{\text{polytime S}} \text{3SAT assignment } \alpha'$$

Pictorially:

Algorithm R: The intuition behind this algorithm is that when we have a clause $(\ell_0 \vee \ell_1 \vee ... \vee \ell_{k-1})$ in the SAT instance φ (with too-large width $k > 3$), we want to break it into multiple clauses of width ≤ 3.

```
R(φ):
Input  : A CNF formula φ
Output : A CNF formula φ' with each clause of width 3
φ' = φ
i = 0
while φ' has a clause C = (ℓ_0 ∨ ... ∨ ℓ_{k-1}) of width k > 3 do
    Remove C
    Add clauses _____________________________________
    i = i + 1
return φ'
```

Algorithm 3.1: R

Note that φ' is *not* an equivalent formula to φ. While φ is on variables $z_0, \ldots z_{n-1}$, the formula φ' is on variables $z_0, \ldots z_{n-1}, y_0, \ldots y_{t-1}$, where t is the number of iterations of the while loop.

Algorithm S: Given an assignment $\alpha' = (\alpha_0, \ldots, \alpha_{n-1}, \beta_0, \ldots, \beta_{t-1})$ to φ', the algorithm simply takes the first n bits, i.e. $\alpha = (\alpha_0, \ldots, \alpha_{n-1})$.

Next we consider the runtime and correctness of the overall reduction algorithm.

Runtime of the reduction algorithm: We first consider the runtime of the algorithm R:

Then, we consider the runtime of the algorithm S, which is simply $O(n)$. Overall, the runtime of the reduction algorithm is $O(nm)$.

Proof of correctness: We need to show that if φ is satisfiable, then the reduction algorithm produces a satisfying assignment, and that if φ is unsatisfiable, the reduction algorithm outputs $\perp$. This proof relies on the following two claims.

Claim 3.3. *If φ is satisfiable then $\varphi' = R(\varphi)$ is satisfiable.*

Proof sketch.

□

Claim 3.4. *If α' satisfies $\varphi' = R(\varphi)$, then $\alpha = S(\alpha')$ also satisfies φ.*

Proof sketch.

□

To finish the correctness proof, suppose φ is satisfiable. Then from Claim 3.3, φ' is also satisfiable. The 3SAT oracle returns a satisfying assignment α', which is turned into a satisfying assignment for φ via the algorithm S (Claim 3.4). If φ is unsatisfiable, then by Claim 3.4, φ' is also unsatisfiable. In this case, the 3SAT oracle returns $\perp$, so the reduction algorithm also returns $\perp$.

This completes the proof that 3SAT is $\mathsf{NP}_{\mathsf{search}}$-complete.

4 Mapping Reductions

The usual strategy for proving that a problem Γ in $\mathsf{NP}_{\mathsf{search}}$ is also $\mathsf{NP}_{\mathsf{search}}$-hard (and hence $\mathsf{NP}_{\mathsf{search}}$-complete) follows a structure similar to the proof of Theorem 3.2.

1. Pick a known $\mathsf{NP}_{\mathsf{search}}$-complete problem Π to try to reduce to Γ.
2. Come up with an algorithm R mapping instances x of Π to instances $R(x)$ of Γ.
3. Show that R runs in polynomial time.
4. Show that if x has a valid output, then so does $R(x)$.
5. Conversely, show that if $R(x)$ has an answer, then so does x. Moreover, we can transform valid answers to $R(x)$ into valid answers to x through a *polynomial time* algorithm S.

Reductions with the structure outlined above are called *polynomial-time mapping reductions*, and they are what are typically used throughout the theory of NP-completeness. A formal definition is given in the textbook.

5 Independent Set

Next we turn to Independent Set, specifically the Independent Set–Threshold Search version.

Theorem 5.1. Independent Set *is* $\mathsf{NP}_{\mathsf{search}}$*-complete.*

Proof. We'll do this proof less formally than we did the proof of $\mathsf{NP}_{\mathsf{search}}$-completeness of 3SAT.

1. In $\mathsf{NP}_{\mathsf{search}}$:

2. $\mathsf{NP}_{\mathsf{search}}$-hard: We will show 3SAT $\leq_p$ Independent Set.

Note that, whereas we've previously encoded many other problems in SAT, here we're going in the other direction and showing that SAT (or logic) can be encoded by a graph problem!

Our reduction $R(\varphi)$ takes in a CNF formula and produces a graph G and a size k. We'll use as a running example the formula

$$\varphi(z_0, z_1, z_2, z_3) = (\neg z_0 \vee \neg z_1 \vee z_2) \wedge (z_0 \vee \neg z_2 \vee z_3) \wedge (z_1 \vee z_2 \vee \neg z_3).$$

Our graph G consists of:

- Variable gadgets:

- Clause gadgets:

- Conflict edges:

We pick $k = m + n$, which implies that every independent set of size k must contain exactly one vertex from each variable gadget and one vertex from each clause gadget. An algorithm R can create this graph (and k) in polynomial time given φ. The graph for the formula φ is below.

Claim 5.2. *G has an independent set of size $k = n+m$ if and only if φ is satisfiable. Moreover, we can map independent sets in G of size k to satisfying assignments of φ in polynomial time.*

Proof of claim.

□

This completes the proof that Independent Set is $\mathsf{NP}_{\mathsf{search}}$-complete. □

6 Search vs. Decision

Q: Does the seeming hardness of $\mathsf{NP}_{\mathsf{search}}$ come from the fact that there are many possible answers? What if we restrict to decision problems, with only two possible answers?

Definition 6.1. A computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$ is a *decision problem* if $\mathcal{O} =$ _____
and for every $x \in \mathcal{I}$, $|f(x)| =$ _____

By definition,

$$\begin{aligned} \mathsf{P} &= \underline{\hspace{6cm}} \\ \mathsf{EXP} &= \underline{\hspace{6cm}} \end{aligned}$$

However, the decision class NP has a more subtle definition in terms of $\mathsf{NP}_{\mathsf{search}}$:

Definition 6.2 (NP)**.** A decision problem $\Pi = (\mathcal{I}, \{\texttt{yes}, \texttt{no}\}, f)$ is in NP if there is a computational problem $\Gamma = (\mathcal{I}, \mathcal{O}, g) \in \mathsf{NP}_{\mathsf{search}}$ such that for all $x \in \mathcal{I}$, we have:

$$\begin{aligned} f(x) = \{\texttt{yes}\} \quad &\Leftrightarrow \quad \underline{\hspace{4cm}} \\ f(x) = \{\texttt{no}\} \quad &\Leftrightarrow \quad \underline{\hspace{4cm}} \end{aligned}$$

Example 6.3. Let's see a couple of examples of problems in NP.

-

-

Another appealing interpretation of the class NP:

Pursuing this viewpoint, it turns out that there is a deep connection between mathematical proofs and NP, and this is one reason that the P vs. NP question is considered to be a central open problem in mathematics as well as computer science.

One nice feature of focusing on decision problems is that we can show that NP contains P:

Lemma 6.4. $\mathsf{P} \subseteq \mathsf{NP}$.

Proof.

□

In contrast, as shown in the problem set,$\mathsf{P}_{\mathsf{search}}$ is *not* a subset of $\mathsf{NP}_{\mathsf{search}}$, since $\mathsf{NP}_{\mathsf{search}}$ requires that *all* solutions are short and easy to verify, whereas $\mathsf{P}_{\mathsf{search}}$ only tells us that at least one of the solutions is easy to find.

The "P vs. NP Question" is usually formulated as asking whether $\mathsf{P} = \mathsf{NP}$ (with the answer widely conjectured to be no). It turns out that this decision formulation is equivalent to the search version we have been studying:

Theorem 6.5 (Search vs. Decision)**.** $\mathsf{NP} = \mathsf{P}$ *if and only if* $\mathsf{NP}_{\mathsf{search}} \subseteq \mathsf{P}_{\mathsf{search}}$.

Q: Does this theorem remind you of anything you've seen?

A:

Proof of Theorem 6.5. Suppose that $\mathsf{NP}_{\mathsf{search}} \subseteq \mathsf{P}_{\mathsf{search}}$.

For the converse, assume that $\mathsf{P} = \mathsf{NP}$. Since SAT–Decision is in NP, it is also in P and hence in $\mathsf{P}_{\mathsf{search}}$. By Lemma 6.6 below, SAT $\leq_p$ SAT–Decision, so SAT is also in $\mathsf{P}_{\mathsf{search}}$. Since SAT is $\mathsf{NP}_{\mathsf{search}}$-complete, we have $\mathsf{NP}_{\mathsf{search}} \subseteq \mathsf{P}_{\mathsf{search}}$.

□

Lemma 6.6. SAT $\leq_p$ SAT–Decision.

Proof.

```
SATDecisionToSAT(φ):
Input  : A CNF formula φ(x_0, ..., x_{n-1}) (and access to an oracle
         O solving SAT–Decision)
Output : A satisfying assignment α to φ, or ⊥ if none exists.
if O(φ) = no then return ⊥;
foreach i = 0, ..., n − 1 do
    if O(_____________________________) = yes then α_i = 0;
    else α_i = 1;
return α = (α_0, ..., α_{n-1})
```

Algorithm 6.1: Reduction from SAT–Decision to SAT

Runtime:

Correctness:

□

In most textbooks, the theory of NP-completeness focuses on decision problems (and names like SAT, Graph Coloring, etc. typically refer to the decision versions). As expected, we say a decision problem Π is NP-*complete* if $\Pi \in \mathsf{NP}$, and Π is NP-hard, meaning $\Gamma \leq_p \Pi$ for every problem $\Gamma \in \mathsf{NP}$. All of the $\mathsf{NP}_{\mathsf{search}}$-completeness proofs we have seen are also NP-completeness proofs of the corre-

sponding decision problems, since they all use mapping reductions, which have the property that $R(x)$ has a solution iff x has a solution:

Theorem 6.7. SAT–Decision, 3SAT–Decision, Independent Set–Threshold Decision, Subset Sum–Decision, *and* Long Path–Decision *are* NP*-complete.*

For NP-completeness proofs as in the above theorem, mapping reductions become even simpler; we only need the polynomial-time algorithm R that transforms `yes` instances to `yes` instances, and `no` instances to `no` instances. We don't need the algorithm S that maps solutions to the search problem on $R(x)$ back to solutions to the search problem on x.